

Supplementary Information

A Additional Results

Additional segmentation results to the ones presented in Fig. 5B are shown in Fig. S1.

Results complementary to Fig. 3 are presented in Fig. S2.

B ORGAN Implementation Details

Each model is trained for 200 epochs. The learning rate of both generators is 0.0001 and is kept constant for the first 100 epochs and linearly decreased to 0 during the next 100 epochs. The learning rate for the discriminators has a linear warm-up of 15 epochs before adopting the same schedule as the generators. For each iteration, the weights of both generators are updated first. Then, the two discriminators are trained using data from the last two batches for *Tetrominoes* and the last five batches for the remaining datasets. The noise fed to the image generator is a grayscale image sampled from a Gaussian distribution. The generated noise image is 16 times smaller than the input image and upsampled to the same size. For the feature network, the noise is generated in the same way, except that the size is equivalent to the patch size. The noise of the top-k operation is kept constant at 0.02 for the first 100 epochs and linearly decreases to 0 for the remaining 100 epochs. The empirical mean used for the differentiable top-k method is estimated by 250 independent samples. For evaluation, the top-k step is replaced by a simple hard-max function to save memory. Additional hyperparameter choices can be found in Table S1, S2, S3, S4, S5, and S6. All models, including the baseline models with which we benchmarked ORGAN, were trained on a GPU with 11 GB of memory. The total training time of a model was between 27 and 45 h.

C Baselines Implementation Details

For SPACE, the official implementation found on <https://github.com/zhixuan-lin/SPACE> is used. The hyperparameters used for the different datasets are found in Table S7. The original implementation does not directly allow training on images of 256×256 . To circumvent this, the mask decoder of the background module is adapted to dynamically deal with input sizes larger than 128×128 . For *Tetrominoes*, the images are rescaled to 128×128 .

The SLATE implementation from <https://github.com/singhgautam/slate> is used. The model is trained with a batch size of 16 and for 800 epochs. Usually, a model already converged after roughly 25 epochs. The number of slots is again set to the maximal number of objects of the dataset plus one. The learning rate is

set to 0.0003. The remaining settings are kept at the default values. The images of *Tetrominoes* were rescaled to 128×128 to fit the architecture.

For SPOT, the implementation from <https://github.com/gkakogeorgiou/spot> with mostly default implementation parameters is used, except for the following parameters. The number of slots is set to the maximum number of objects plus one. For all datasets, the images are rescaled to 224×224 . The batch size is set to 128. Both the teacher and the student models are trained for 200 epochs. The learning rate follows a cosine decay schedule, starting from $4 \cdot 10^{-6}$ and gradually decreasing to $4 \cdot 10^{-8}$ using a cosine scheduler over the course of training.

The LSD model is taken from <https://github.com/JindongJiang/latent-slot-diffusion/tree/master> and used with default implementation parameters except for the following. The number of slots varied from 4 for the *Tetrominoes* dataset to 12 for all other datasets to correspond to the number of objects per image. The images of all datasets are resized to 256×256 . The batch size is reduced to 8 due to memory constraints. The model is trained on the *Multi-MNIST* dataset for 150,000 training steps, while for the *Cells* and *Sprites* dataset, the model is trained for 100,000 training steps. For the *Tetrominoes* dataset, the model is trained for 75,000 training steps. The learning rate is fixed at $4 \cdot 10^{-4}$. In Fig. S3, the processing of the *Tetrominoes* dataset by LSD is presented. The VAE reconstruction column accurately reproduces the input images, keeping the shapes and colors of the blocks intact. This shows that the VAE effectively captures the important features of the input. Objects with the same color are assigned to the same slots. In addition, dark blue objects are attributed to the background.

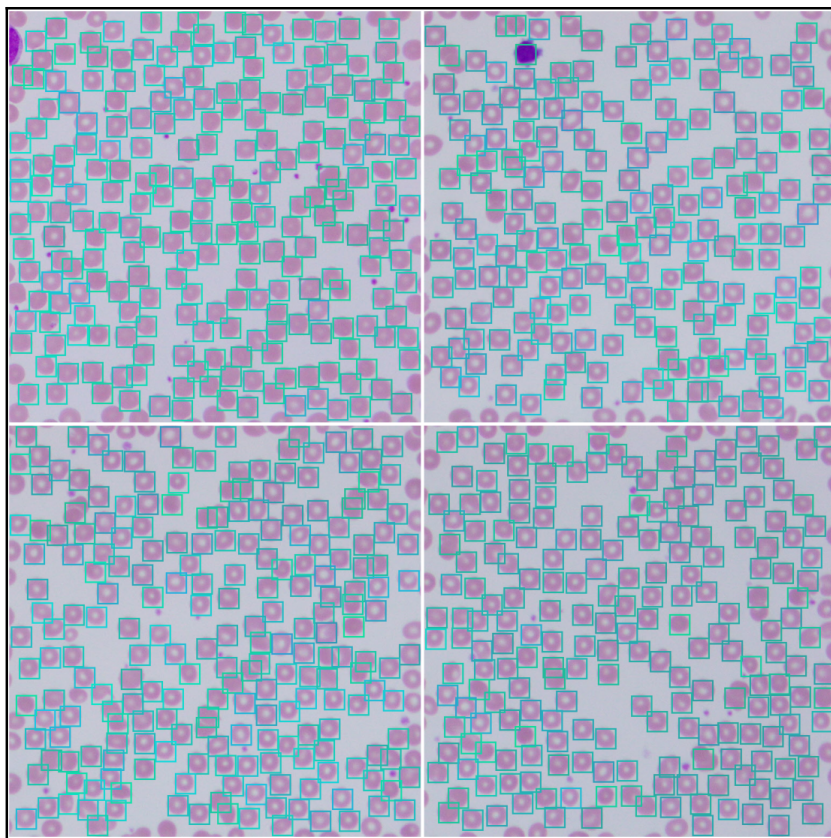


Fig. S1. Additional examples for the generalization capabilities of ORGAN on the *Cells* dataset with 768×768 images. The subfigures are similar to Fig. 5B.

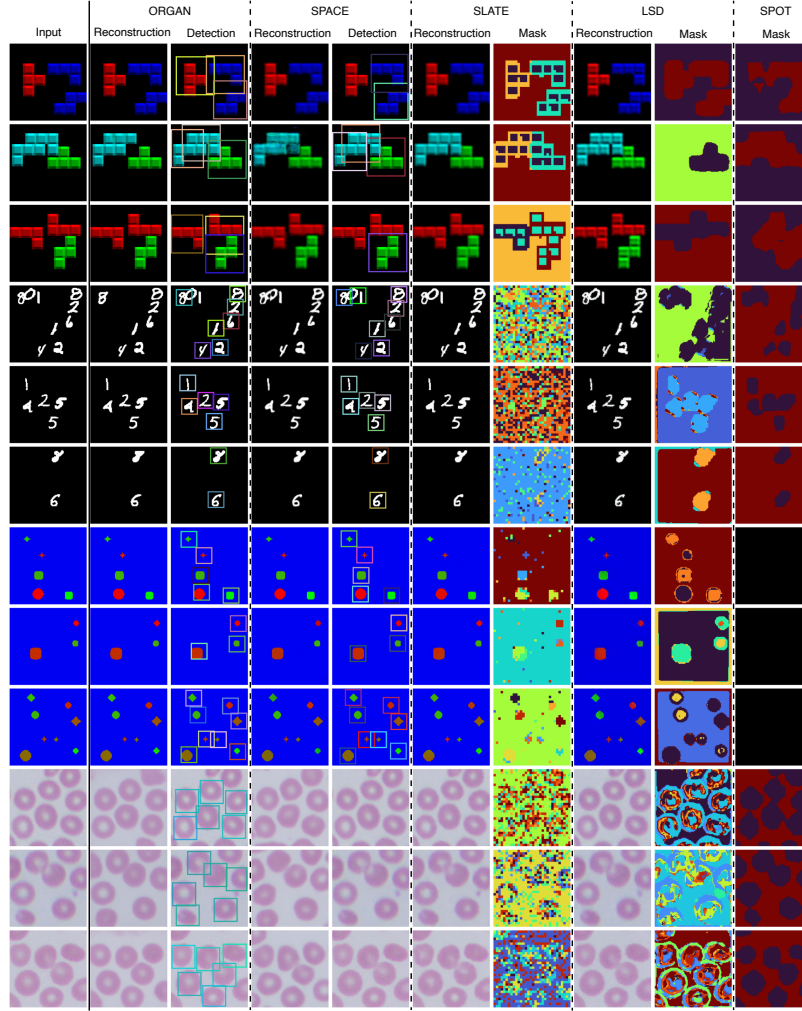


Fig. S2. Additional examples of object detection performance for the four datasets. This figure is similar to Fig. 3.

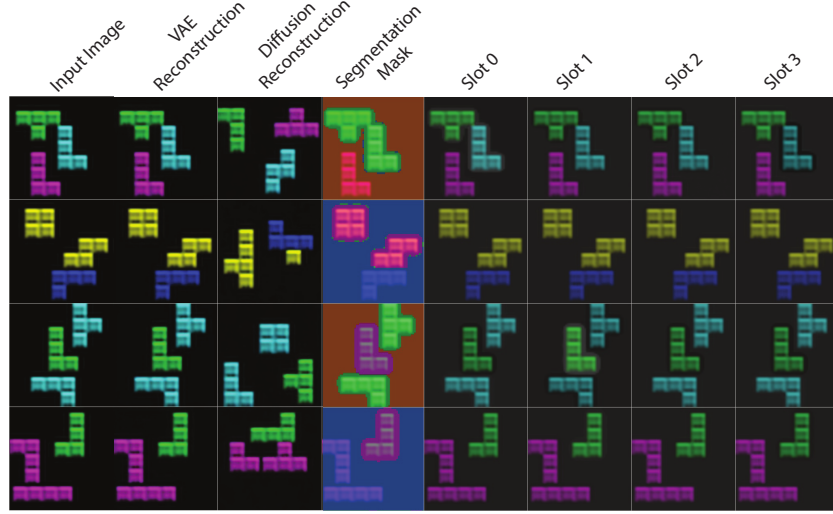


Fig. S3. Image reconstruction and object segmentation using a Latent Slot Diffusion model. Each row shows a different tetrominoes configuration processed through the model pipeline. From left to right: original input image, VAE reconstruction, diffusion reconstruction, segmentation mask with color-coded object classes, and individual object slots (0-3) showing the decomposition of the scene.

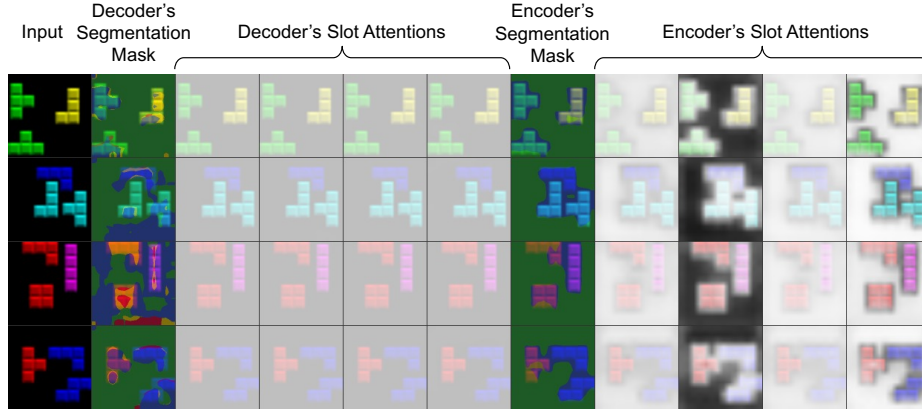


Fig. S4. Visualization of SPOT's representations on the *Tetrominoes* dataset. Each row corresponds to a different input image. From left to right: input image, predicted segmentation masks overlaid on the input, decoder's slot attention maps, segmentation masks from the slot-based encoder, and its corresponding slot attention maps.

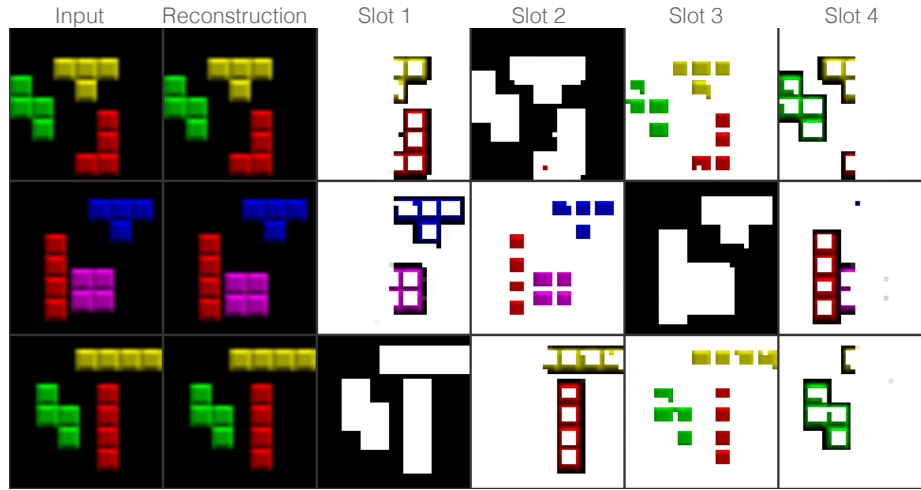


Fig. S5. Illustration of SLATE’s representations on the *Tetrominoes* dataset. Each row corresponds to a different input image. From left to right: input image, reconstruction, and cutouts of the original image based on the attention of the slot.

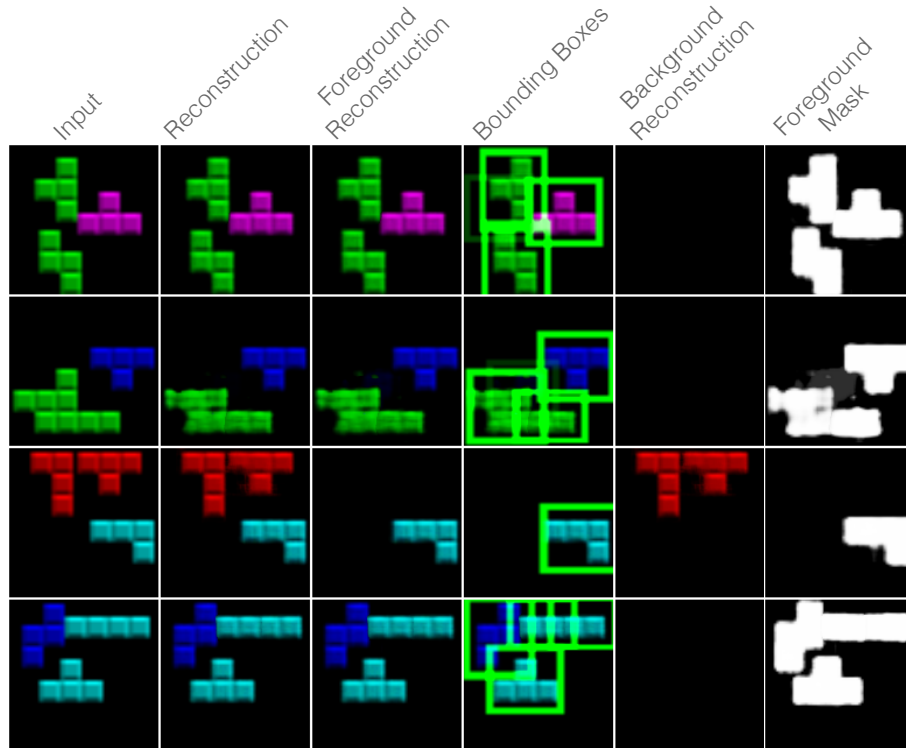


Fig. S6. Visualization of the utilization of the different components of SPACE on *Tetrominoes*. Each row corresponds to a different input image. From left to right: input image, reconstruction, reconstruction with the foreground module, reconstruction of the foreground module combined with bounding boxes of the objects, reconstruction with the background module, and foreground mask.

D Dataset Details

The value range for images and the feature vector of a list element is restricted to $[0, 1]$.

The number of objects placed in an image of size 128×128 for *Sprites* follows the binomial distribution $B(n, p)$ with $p = 0.5$ and $n = 10$. For the image size of 256×256 and 512×512 , n is set to 40 and 160 respectively. The colors of the objects span from red to green. The shapes range from squares to stars of various scales. The training set consists of 10,000 images and 10,000 lists, where 1,000 samples of each domain are used for validation. The distance between the centers of objects is set to be at least 18 pixels. The shape, color, and size of an object are sampled uniformly.

For *Multi-MNIST*, the number of digits follows a discrete uniform distribution $\mathcal{U}_{\{1,10\}}$. The digits are placed in a 128×128 image. Their location is sampled with the Poisson-disk algorithm. Objects may be partially overlapping. Input lists are generated in a similar way to *Sprites*. The feature vector η is uniformly sampled and does not have a direct connection to the used digits. The number of training images and lists is 10,000 each. The validation split is equivalent to *Sprites*. The spacing between the centers of digits is at least 15 pixels.

For *Tetrominoes*, 100,000 images are used for training and validation, and an additional 100,000 for testing. The list is generated with α being fixed to 0.99. The locations are generated by randomly permuting all x and y coordinates of the ground truth in the input images.

In *Cells*, the original images are sliced to the size of 128×128 with a stride of 64. For the cutout of 768×768 , a stride of 384 is used. All cutouts of a single original image are either in the test or train set. The training/validation set comprises 9,135 patches, following the same split as the *Sprites* dataset. For the list generation, object centers are at least 25 pixels apart and the count is sampled from $B(10, 0.5)$. The number of lists in the training and validation sets is equivalent to the number of images. Expert-wise F1-scores are shown in Fig. S7.

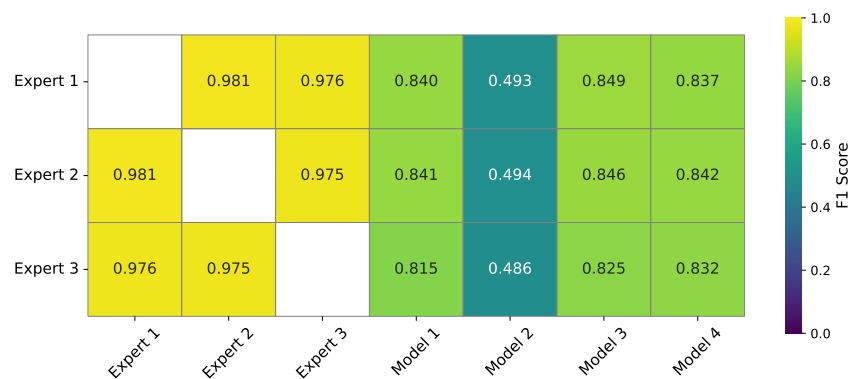


Fig. S7. Pairwise F1-scores between human annotators and model predictions for the *Cells* dataset. The matrix shows F1-scores between each expert and four model instances. The average inter-annotator F1-score was 0.977, while models achieved an average F1 of 0.750.

E Architecture Details

Table S1. Score network s_ϕ model architecture.

Layer	Size/channels	Stride/dilation	Normalization/activation
Input	c	–	–
Conv 3×3	24	2/1	leaky ReLU, dropout
Conv 3×3	24	2/1	leaky ReLU, dropout
Conv 3×3	24	1/2	leaky ReLU, dropout
Global avg pool	–	–	–
Linear	1	–	–

Table S2. Feature network f_θ model parameters.

Layer	Size/channels	Kernel	Normalization/activation
Input	$c + 1$	–	–
Conv	64	3×3	GroupNorm (4 groups), leaky ReLU, dropout
Max pool	–	2×2 , Stride 2	–
Conv	128	3×3	GroupNorm (4 groups), leaky ReLU, dropout
Max pool	–	2×2 , Stride 2	–
Conv	256	3×3	GroupNorm (4 groups), leaky ReLU, dropout
Max pool	–	2×2 , Stride 2	–
Conv	512	3×3	GroupNorm (4 groups), leaky ReLU, dropout
Max pool	–	2×2 , Stride 2	–
Global avg pool	256	1×1	–
Linear (FC1)	256	–	Leaky ReLU, dropout
Linear (FC2)	256	–	Leaky ReLU, dropout
Linear (output)	$2 + 1 + \eta $	–	Tanh/sigmoid/sigmoid

Table S3. U-Net u_ψ model architecture.

Layer	Size/channels	Kernel	Normalization/activation	Connections
Input	$ \eta + 2$	–	–	–
Encoder block 1:				
Conv	64	3×3	GroupNorm (4 groups), leaky ReLU	
Conv	64	3×3	GroupNorm (4 groups), leaky ReLU	Decoder block 2
Max pool	–	2×2 , stride 2	–	–
Encoder block 2:				
Conv	128	3×3	GroupNorm (4 groups), leaky ReLU	
Conv	128	3×3	GroupNorm (4 groups), leaky ReLU	Decoder block 2
Max pool	–	2×2 , stride 2	–	–
Center block:				
Conv	256	3×3	GroupNorm (4 groups), leaky ReLU	
Conv	128	3×3	GroupNorm (4 groups), leaky ReLU	
Decoder block 1:				
Max unpool	–	2×2 , stride 2	–	Encoder block 2
Conv	128	3×3	GroupNorm (4 groups), leaky ReLU	
Conv	64	3×3	GroupNorm (4 groups), leaky ReLU	
Decoder block 2:				
Max unpool	–	2×2 , stride 2	–	Encoder block 1
Conv	64	3×3	GroupNorm (4 groups), leaky ReLU	
Conv	64	3×3	GroupNorm (4 groups), leaky ReLU	
Out block:				
Conv	64	1×1	GroupNorm (4 groups), leaky ReLU	
Conv	64	1×1	GroupNorm (4 groups), leaky ReLU	
Conv	c	1×1	–	

Table S4. List discriminator $D_L(\cdot; \xi_L)$ model architecture.

Layer	Size/channels	Normalization/activation
Input	$k \times (3 + \eta)$	–
Linear	24	ReLU, spectral norm, dropout
Transformer layer	24	Dropout
Transformer layer	24	Dropout
Transformer layer	24	Dropout
Linear	24	ReLU, spectral norm, dropout
Linear	24	ReLU, spectral norm, dropout
Linear	1	spectral norm
Global mean pool	1	–

Table S5. Image discriminator $D_I(\cdot; \xi_I)$ model architecture.

Layer	Size/channels	Kernel/pooling	Normalization/activation
Input	im_dim	–	–
Conv block 1	16	3×3	Spectral norm, dropout
Max pool	–	2×2	–
Conv block 2	32	3×3	Spectral norm, dropout, leaky ReLU
Max pool	–	2×2	–
Conv block 3	64	3×3	Spectral norm, dropout, leaky ReLU
Max pool	–	2×2	–
Conv block 4	128	3×3	Spectral norm, dropout, leaky ReLU
Max pool	–	2×2	–
Final conv	1	3×3	Spectral norm
Global mean pool	–	–	–

Table S6. Hyperparameters of ORGAN.

Parameter	Value
Global hyperparameters:	
Optimizer	Adam
Initial learning rate	0.0001
λ_{loc}	0.5
λ_{Cyc_I}	100
λ_{Cyc_L}	10
λ_{Dis_I}	1
λ_{Dis_L}	1
λ_{pres}	15
n_{warmup}	15
Epochs	200
Dataset-specific hyperparameters:	
<i>Tetrominoes:</i>	
Batch size	40
p_w	20
p_h	20
Stride	3
σ	2
$ \eta $	8
n_{pad}	3
<i>Multi-MNIST:</i>	
Batch size	6
p_w	28
p_h	28
Stride	4
σ	2.8
$ \eta $	16
n_{pad}	3
<i>Sprites:</i>	
Batch size	6
p_w	28
p_h	28
Stride	4
σ	2.8
$ \eta $	3
n_{pad}	3
<i>Cells:</i>	
Batch size	6
p_w	48
p_h	48
Stride	6
σ	4.8
$ \eta $	2
n_{pad}	0

Table S7. Hyperparameters for SPACE. Hyperparameters that the original publication kept fixed and remained unchanged in this work are not listed.

Dataset	<i>Multi-MNIST</i>	<i>Sprites</i>	<i>Tetrominoes</i>	<i>Cells</i>
Training parameters				
Batch size	10	12	12	12
Max epochs	1000	1000	1000	1000
Max steps	160000	160000	160000	160000
Architecture parameters				
Grid size G	8	16	8	8
Foreground std	0.15	0.20	0.15	0.15
Background std	0.15	0.10	0.15	0.15
z^{pres} prior prob	(0.1 $\rightarrow$ 0.01) (4000 $\rightarrow$ 10000)	(0.1 $\rightarrow$ 1e-10) (10000 $\rightarrow$ 15000)	(0.1 $\rightarrow$ 0.01) (4000 $\rightarrow$ 10000)	(0.1 $\rightarrow$ 0.01) (4000 $\rightarrow$ 10000)
z^{scale} prior mean	(-0.5 $\rightarrow$ -1.0) (10000 $\rightarrow$ 20000)	(-2.0 $\rightarrow$ -2.5) (0 $\rightarrow$ 20000)	(0.3 $\rightarrow$ -0.3) (10000 $\rightarrow$ 20000)	(-0.4 $\rightarrow$ -0.8) (10000 $\rightarrow$ 20000)
Gumbel-Softmax temperature τ	(2.5 $\rightarrow$ 0.5) (0 $\rightarrow$ 20000)	(2.5 $\rightarrow$ 2.5) (0 $\rightarrow$ 10000)	(2.5 $\rightarrow$ 0.5) (0 $\rightarrow$ 20000)	(2.5 $\rightarrow$ 0.5) (0 $\rightarrow$ 20000)
#Steps to fix α	—	4000	—	—
Fix α value	—	0.1	—	—
Component number K	1	3	1	1
Boundary loss	True	False	True	True
BL off step	100000	—	100000	100000

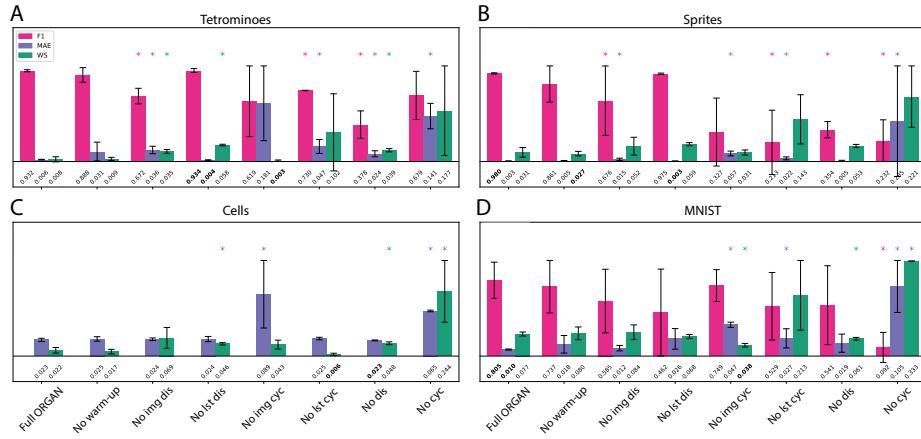


Fig. S8. Ablation study for the four datasets presented in the paper. The performance of different parts of the network is tested using the F1-score (higher is better), MAE (lower is better), and Wasserstein loss (lower is better). Both the F1-score and the Wasserstein loss are applied to the lists, while MSE is applied to the images. For each condition, the standard deviation is plotted. Since there is no ground-truth data for *Cells*, no F1-score can be calculated for it. For each condition, four networks have been trained. Significance was tested using a two-sided Mann-Whitney U-test (* : $p \leq 0.05$) with respect to the complete ORGAN.

F Ablation Studies

In Fig. S8 we present an ablation study for the four datasets. The ablation study tested the influence of the different loss terms of ORGAN, as well as the effect of warm-up on the performance of the network. For performance evaluation, an F1-score on the presence and location of the object in the list and the MAE between the images is used, so as to cover both the list and the image domain. Finally, the Wasserstein (WS) loss is calculated for the feature vectors. For each run, all three losses are clipped between 0 and 1.

The most important contribution to the MAE is the presence of the image cycle loss. Similarly, the list cycle loss is important for a good F1-score. The lack of either or both discriminators is negatively affecting the performance of ORGAN. While the removal of individual components can occasionally slightly improve specific metrics, their joint inclusion leads to more balanced performance across evaluation criteria. Furthermore, omitting the warm-up period leads to increased variance in the performance of the final models.

We also investigate the effect of the patch size and stride of ORGAN on the *Sprites* dataset. We further investigate the relevance of having a non-homogeneous background. Both results are presented in Table S8. For the patch size and stride, we investigate their influence on the overall performance by varying them while keeping the patch size to stride ratio identical to the ratio used in the manuscript. The results show that the best performance can be achieved when the size of

the larger objects is slightly smaller than the patch size. However, ORGAN can still converge towards the solution when the patch size is off by a factor of 25 %. When the patch size is off by a factor of 2, the Sprites dataset cannot be inferred anymore. Two factors potentially play an important role here: a large patch can contain multiple objects, introducing additional complexities to the feature network, hence stifling its performance. Secondly, as the patch size increases, the number of potential patches after the non-maximum suppression decreases, making the instance segmentation of multiple close-by objects more difficult.

For the background investigation, we adapt the *Sprites* dataset by adding a randomly rotated checkerboard pattern to the background. For consistency purposes, the bright fields are kept identical to the original Sprites dataset, while the dark fields are scaled by $1 - \beta$, where β is in the interval of 0 to 1. ORGAN is performing best if the background is homogeneous. For $\beta \neq 0$, trained instances of ORGAN can still perform well, as can be seen from the recorded ranges of the F1-score in Table S8. However, the training procedure becomes inherently unstable. Furthermore, the best-performing networks tend to perform worse as β increases. The exception is at $\beta = 0.1$, where none of the 4 instances could be trained in a stable manner. The network started to reconstruct the global background instead of the local foreground objects for larger values of β .

Table S8. Ablation studies on the *Sprites* dataset. **A:** Effect of patch size on the performance of ORGAN in the list domain. The stride was increased in correlation with the patch size, as in the original manuscript. **B:** Effect of a non-homogeneous checkerboard background. Ranges indicate the worst- and best-performing model across runs ($n = 4$).

A: Patch size ablation						
Patch size / stride	F1	F1 range	MAE	MAE range	WS	WS range
21 / 3	0.496 ± 0.492	0.002–0.993	0.017 ± 0.013	0.003–0.030	0.057 ± 0.041	0.018–0.122
28 / 4	0.980 ± 0.006	0.971–0.987	0.003 ± 0.001	0.002–0.004	0.031 ± 0.016	0.013–0.057
35 / 5	0.502 ± 0.358	0.048–0.957	0.011 ± 0.011	0.004–0.030	0.026 ± 0.007	0.015–0.033
42 / 6	0.069 ± 0.043	0.012–0.120	0.030 ± 0.000	0.030–0.030	0.018 ± 0.005	0.008–0.022
49 / 7	0.078 ± 0.035	0.022–0.115	0.030 ± 0.000	0.030–0.030	0.025 ± 0.013	0.010–0.041
B: Checkerboard background ablation						
β	F1	F1 range	MAE	MAE range	WS	WS range
0.000	0.980 ± 0.006	0.971–0.987	0.003 ± 0.001	0.002–0.004	0.031 ± 0.016	0.013–0.057
0.025	0.447 ± 0.446	0.000–0.952	0.023 ± 0.011	0.011–0.033	0.023 ± 0.013	0.005–0.042
0.100	0.022 ± 0.014	0.000–0.035	0.038 ± 0.004	0.033–0.042	0.073 ± 0.088	0.009–0.224
0.250	0.496 ± 0.481	0.000–0.977	0.041 ± 0.020	0.021–0.074	0.018 ± 0.019	0.006–0.051
0.500	0.435 ± 0.416	0.020–0.905	0.025 ± 0.007	0.015–0.033	0.014 ± 0.011	0.001–0.026
1.000	0.351 ± 0.356	0.004–0.904	0.106 ± 0.086	0.021–0.210	0.044 ± 0.035	0.010–0.092

We analyze the influence of the differentiable top-k operator by replacing it with a non-differentiable hardmax selection. As shown in Table S9, this modification has only a limited effect on the final performance. Nevertheless, we retain the differentiable top-k formulation during training because it provides a continuous relaxation of the discrete object-selection step. This allows gradients to propagate through the scoring operation and yields a fully differentiable end-to-end objective, rather than relying on a fixed hard assignment during optimization.

Table S9. Ablation study of the score network. The differentiable score network used in the complete ORGAN model was replaced with a non-differentiable score network.

Metric	Model	<i>Tetrominoes</i>	<i>Sprites</i>	<i>Multi-MNIST</i>
F1	Full ORGAN	0.932 ± 0.011	0.980 ± 0.006	0.805 ± 0.193
	Non-differentiable score network	0.920 ± 0.018	0.921 ± 0.044	0.889 ± 0.071
MAE	Full ORGAN	0.006 ± 0.001	0.003 ± 0.001	0.010 ± 0.001
	Non-differentiable score network	0.007 ± 0.001	0.001 ± 0.000	0.009 ± 0.003
WS	Full ORGAN	0.008 ± 0.009	0.031 ± 0.016	0.077 ± 0.007
	Non-differentiable score network	0.003 ± 0.001	0.051 ± 0.025	0.082 ± 0.014